

Lucas' Theorem &

PS Study Notes

2026-04-04

Table of contents

1		2
2	: Lucas' Theorem	2
2.1	Statement	2
2.2	$p = 2$ ()	2
2.3	Submask ?	3
2.4	(GF(2))	3
3		3
3.1		4
4		4
4.1		4
4.2		5
5	Popcount	5
5.1	(x86-64 POPCNT)	5
5.2	(Hamming Weight)	5
5.3	$O(1)$	6
6		6
6.1	1: popcount	6
6.2	2:	6
6.3	3:	6
6.4	4: $n \leq 1$	7
7		7
7.1	Kummer's Theorem	7
7.2	p	7

1

N , $N + 1$

$$\binom{N}{0}, \binom{N}{1}, \dots, \binom{N}{N}$$

.

- $0 \leq N \leq 10^{18}$
- 100,000

2 : Lucas' Theorem

2.1 Statement

p , $N \leq k \leq p$:

$$\binom{N}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}$$

$$N = \sum n_i p^i, k = \sum k_i p^i.$$

2.2 $p = 2$ ()

$$\binom{0}{1} = 0 :$$

$$\binom{N}{k} \iff k \leq N \iff k \leq N \text{ binary submask}$$

2.3 Submask ?

$(k \& N) == k$ $\iff k$ N submask .
 $, k$ 1 N 1 .

$N = 1010$ $k = 0010 \rightarrow (k \& N) = 0010 = k \rightarrow$ submask $\rightarrow$
 $N = 1010$ $k = 0001 \rightarrow (k \& N) = 0000 \neq k \rightarrow$ not submask $\rightarrow$

2.4 (GF(2))

GF(2) Freshman's dream:

$$(1+x)^{2^i} \equiv 1+x^{2^i} \pmod{2}$$

$$N = \sum_i n_i \cdot 2^i \quad :$$

$$(1+x)^N = \prod_{i: n_i=1} (1+x)^{2^i} \equiv \prod_{i: n_i=1} (1+x^{2^i}) \pmod{2}$$

$$x^k \quad x^{2^i} \quad \rightarrow \quad \text{submask} \quad .$$

3

$$= 2^{\text{popcount}(N)}$$

$$\boxed{= (N+1) - 2^{\text{popcount}(N)}}$$

$$\text{popcount}(N) \leq N \quad 1 \quad .$$

N	$N ()$	popcount	(2^p)	$(N + 1)$
-----	---------	----------	---------	-----------

3.1

N	N ()	popcount	(2^p)	$(N + 1)$	
0	0	0	1	1	0
1	1	1	2	2	0
4	100	1	2	5	3
10	1010	2	4	11	7
100	1100100	3	8	101	93

4

4.1

```
#include <stdio.h>
using ull = unsigned long long int;

//      popcount (Hamming Weight      )
ull popcount_sw(ull x) {
    x = x - ((x >> 1) & 0x5555555555555555ULL); // 2
    x = (x & 0x3333333333333333ULL)
        + ((x >> 2) & 0x3333333333333333ULL); // 4
    x = (x + (x >> 4)) & 0x0F0F0F0F0F0F0F0FULL; // 8
    return (x * 0x0101010101010101ULL) >> 56; //
}

int main(void)
{
    int t; scanf("%d", &t);
    for (int tc = 1; tc <= t; ++tc)
    {
        ull n; scanf("%llu", &n);

        ull p      = popcount_sw(n); // N
```

```

ull odd_cnt = 1ULL << p; // = 2^p
ull ans = (n + 1ULL) - odd_cnt;

printf("#%d %llu\n", tc, ans);
}
}

```

4.2

(k=0~N)	$O(N)$ per query	$N \leq 10^{18} \rightarrow$	
Lucas + popcount	$O(1)$ per query	POPCNT = 1	

5 Popcount

5.1 (x86-64 POPCNT)

`__builtin_popcountll` , POPCNT CPU .

```
POPCNT rax, rdi ; 1 , 0(1)
```

5.2 (Hamming Weight)

POPCNT bit-parallel . $\log_2 W = 6$ (64) .

```

:   1 0 1 1 0 1 0 0   ( : 8 )
    ↓
1:  01 10 01 00       (2   )
    ↓
2:  0011 0001         (4   )
    ↓
3:  00000100          ( : 1   = 4)

```

. 64 .

5.3 $O(1)$

POPCNT	$O(1)$	1
Hamming Weight	$O(\log W) = O(1)$	~ 6
	$O(W)$	64

$W = 64$ $O(\log W) = O(1)$.

6

6.1 1: popcount

```
// : popcount
ull ans = popcount_sw(n);
printf("#%d %llu\n", tc, n + 1 - ans); // ←

// : 2^popcount
ull odd_cnt = 1ULL << popcount_sw(n);
ull ans      = (n + 1ULL) - odd_cnt; // ←
```

6.2 2:

```
// : 1 << p    p >= 32   int
ull odd_cnt = 1 << p;

// :    1ULL
ull odd_cnt = 1ULL << p;
```

6.3 3:

```
// : ans ull %d
printf("#%d %d\n", tc, ans);

//
printf("#%d %llu\n", tc, ans);
```

6.4 4: $n \leq 1$

```
// ( )
if (n <= 1) { printf("#%d 0\n", tc); continue; }
// n=0: 2^0=1, ans=1-1=0
// n=1: 2^1=2, ans=2-2=0
```

7

7.1 Kummer's Theorem

$\binom{m+n}{m} \equiv p^{\text{carry}}$ (Lucas Kummer)

7.2 p

p , $k_i > n_i$:

$$\binom{N}{k} \equiv 0 \pmod{p}$$

$O(\log_p N)$ per query.

7.3

- BOJ 11401 — $\text{mod } p$ (+)
- BOJ 16977 —
- Lucas + CRT